Laboratorio di Architetture Software e Sicurezza Informatica [AAF1569]

7 - Rails (part 3)

SAPIENZA
UNIVERSITÀ DI ROMA

Part of therein contents are based on slides of Prof. Armando Fox

DIAG
Dipartimento di Ingegneria informatica, automatica e gestionale Antonio Ruberti

Moviegoers and Reviews for RottenPotatoes

```
rails generate model Moviegoer name:string
```

```
# Run 'rails generate migration create_reviews' and then
#   edit db/migrate/*_create_reviews.rb to look like this:
class CreateReviews < ActiveRecord::Migration
  def change
    create_table 'reviews' do |t|
      t.integer    'potatoes'
      t.text       'comments'


    end
  end
end
```

How do we make our three models reference each other? For example, how can we **associate** a Review with a Movie and a Moviegoer?

Associations

An association is a logical relationship between two types of entities (models)

https://guides.rubyonrails.org/v6.1/association_basics.html

https://guides.rubyonrails.org/v6.1/active_record_migrations.html

With Active Record associations we can declare a connection between the two models. In a UML diagram, we would like to have these relationships:

ActiveRecord Associations

We need a way to represent associations for our stored objects. In a relational database, **foreign key** mechanisms are helpful for that. For example:

```
SELECT reviews.*
    FROM movies JOIN reviews ON movies.id=reviews.movie_id
    WHERE movies.id = 41
```

Moving to Rails, we would like to have a mechanism in which each object of a class had “direct references” to its associated objects.

The **`ActiveRecord::Associations`** module supports exactly this design.

ActiveRecord Associations

A **`belongs_to`** association sets up a connection with another model: each instance of the declaring model "belongs to" one instance of the other model. As a result, Rails will maintain *primary key-foreign key* correspondence information between instances of the two models

A **`has_many`** association indicates a one-to-many connection with another model. You'll often find this association on the "other side" of a `belongs_to`.

In a migration, we can add to table `t` a foreign key field (i.e., a field that is the primary key of another table) by using `t.`**`references`** `` `othertable` ``.

Moviegoers and Reviews for RottenPotatoes

review.rb

```ruby
class Review < ActiveRecord::Base
  belongs_to :movie
  belongs_to :moviegoer
end
```

```ruby
class CreateReviews < ActiveRecord::Migration
  def change
    create_table 'reviews' do |t|
      t.integer    'potatoes'
      t.text       'comments'
      t.references 'moviegoer'
      t.references 'movie'
    end
  end
end
```

```ruby
class Movie < ApplicationRecord
    has_many :reviews

class Moviegoer < ApplicationRecord
    has_many :reviews
```

More on ActiveRecord Associations

The ActiveRecord Associations module uses Ruby *metaprogramming* to create instance methods to “traverse” associations by building appropriate DB queries.

The **`has_many`** method implies a *collection* of the owned objects (Reviews). Hence, we can iterate over it using standard collection items (e.g., sort, map, and so on). Behind the scenes, it acts as a method call that adds several instance methods to the class to help manage the association.

The **`belongs_to`** method calls in `review.rb` gives to a Review object a `movie` and a `moviegoer` **instance methods** that look up and return, respectively, the Movie and the Moviegoer to which the Review belongs. Note how, in both cases, the method name is singular: they will return a single object (not an enumerable).

More on ActiveRecord Associations

```ruby
# association1.rb
inception = Movie.where(:title => 'Inception').first
# or also: Movie.find_by(:title => 'Inception')
alice,bob = Moviegoer.find(alice_id, bob_id)
# alice likes Inception, bob less so
alice_review = Review.new(:potatoes => 4)
bob_review   = Review.new(:potatoes => 3)
# a movie has many reviews:
inception.reviews = [alice_review, bob_review]
# a moviegoer has many reviews:
alice.reviews << alice_review
bob.reviews << bob_review
# can we find out who wrote each review?
inception.reviews.map { |r| r.moviegoer.name } # => ['alice','bob']
```

More on ActiveRecord Associations

```ruby
# association1.rb
inception = Movie.where(:title => 'Inception').first
# or also: Movie.find_by(:title => 'Inception')
alice,bob = Moviegoer.find(alice_id, bob_id)
# alice likes Inception, bob less so
alice_review = Review.new(:potatoes => 4)
bob_review   = Review.new(:potatoes => 3)
# a movie has many reviews:
inception.reviews = [alice_review, bob_review]
# a moviegoer has many reviews:
alice.reviews << alice_review
bob.reviews << bob_review
# can we find out who wrote each review?
inception.reviews.map { |r| r.moviegoer.name } # => ['alice','bob']
```

`reviews=` is a method call that sets the **`movie_id`** field for both Alice's and Bob's reviews to link them to the Inception movie

Through-Associations

Since any given Review is associated with both a Moviegoer and a Movie, we could say that there's an **indirect** association between Moviegoers and Movies.

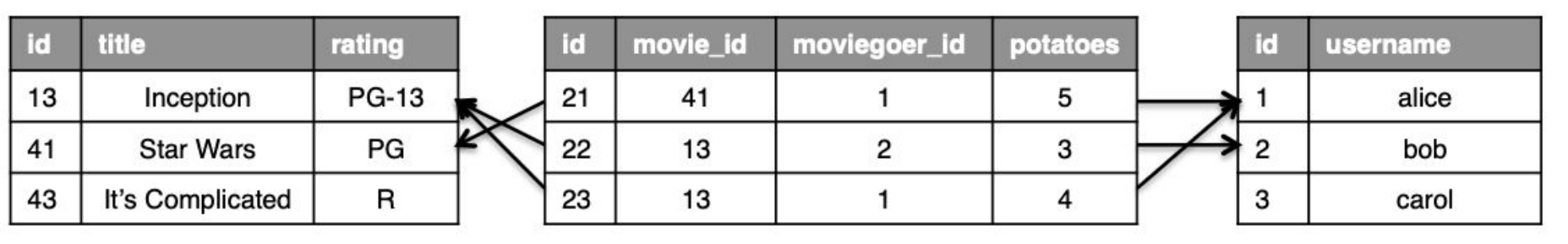
Being this a recurrent pattern, Rails provides an abstraction to simplify its use.

This is called a through-association and is expressed using the `:through` option:

has_many_through_example.rb

```ruby
# in moviegoer.rb:
class Moviegoer
  has_many :reviews
  has_many :movies, :through => :reviews
  # ...other moviegoer model code
end
```

Through-Associations



id	title	rating
13	Inception	PG-13
41	Star Wars	PG
43	It's Complicated	R

id	movie_id	moviegoer_id	potatoes
21	41	1	5
22	13	2	3
23	13	1	4

id	username
1	alice
2	bob
3	carol

```
SELECT moviegoers.*
    FROM moviegoers JOIN reviews ON moviegoers.id = reviews.moviegoer_id
    JOIN movies ON movies.id = reviews.movie_id
    WHERE movies.id = 13
```

Rails will generate efficient SQL join queries for traversing associations

```
13    # can we find out who wrote each review?
14    inception.reviews.map { |r| r.moviegoer.name } # => ['alice','bob']
```

More on ActiveRecord Associations

Associations provide **lifecycle hooks** that trigger when objects are added to or removed from an association (distinct from the lifecycle hooks of the objects)

We can declared **validations** on associated models as well.

Additional options to association methods control what happens to “owned” objects when an “owning” object is destroyed. For example, we can write:

```
has_many: reviews, dependent: destroy
```

which will cause the reviews belonging to a movie to be deleted from the DB if the movie is destroyed.

More on ActiveRecord Associations

validating_associations.rb

```ruby
class Review < ActiveRecord::Base
  # review is valid only if it's associated with a movie:
  validates :movie_id, :presence => true
  # can ALSO require that the referenced movie itself be valid
  #  in order for the review to be valid:
  validates_associated :movie
end
```

Here, a review is only saved if it has been associated with some movie.

RESTful Routes for Associations

How can we refer to actions involving objects from multiple models? Say that we need a way to link a review to a movie and a moviegoer upon creation/update.

Remembering details using `session[]` or similar tricks isn't RESTful...

In our context, we can make the RESTful routes themselves reflect the logical "nesting" of Reviews inside Movies:

nested_routes.rb

```
1  # in routes.rb, change the line 'resources :movies' to:
2  resources :movies do
3    resources :reviews
4  end
```

RESTful Routes for Associations

Helper method	RESTful route and action	
`movie_reviews_path(m)`	`GET /movies/:movie_id/reviews`	index
`movie_review_path(m)`	`POST /movies/:movie_id/reviews`	create
`new_movie_review_path(m)`	`GET /movies/:movie_id/reviews/new`	new
`edit_movie_review_path(m,r)`	`GET /movies/:movie_id/reviews/:id/edit`	edit
`movie_review_path(m,r)`	`GET /movies/:movie_id/reviews/:id`	show
`movie_review_path(m,r)`	`PUT /movies/:movie_id/reviews/:id`	update
`movie_review_path(m,r)`	`DELETE /movies/:movie_id/reviews/:id`	destroy

After the update, these routes will appear **in addition** to the basic RESTful routes on Movies that we have seen in the previous classes.

Rails chooses the outer resource name to make `:movie_id` capture the ID of the "owning" resource, while `:id` will match the ID of the resource (i.e., the review) itself.

RESTful Routes for Associations

```ruby
class ReviewsController < ApplicationController
    before_filter :has_moviegoer_and_movie, :only => [:new, :create]
    protected
    def has_moviegoer_and_movie
      unless (@movie = Movie.where(:id => params[:movie_id]))
        flash[:warning] = 'Review must be for an existing movie.'
        redirect_to movies_path
      end
    end
```

We define a before-filter to ensure that `params[:movie_id]` exists in the DB

Transactions with Active Record objects

Transactions are protective blocks where DB manipulations are only permanent if they can all succeed as if they were one atomic action.

They enforce the integrity of the database and guard the data against program errors: mainly, situations where you have a number of statements that must be executed *together or not at all*. For example, as in the block below:

```
ActiveRecord::Base.transaction do
  david.withdrawal(100)
  mary.deposit(100)
end
```

*Beware: objects will **not** have their **instance data** returned to their pre-transactional state!*

Transactions with Active Record objects

Though the transaction class method is called on some Active Record class, the objects within the transaction block need not all be instances of that class. This is because transactions are per-database connection, not per-model.

```
Account.transaction do
  balance.save!
  account.save!
end
```

Also as a model instance method: e.g., `balance.transaction do [...] end`

https://api.rubyonrails.org/v6.1/classes/ActiveRecord/Transactions/ClassMethods.html

Transactions with Active Record objects

A transaction acts on a single database connection. With multiple class-specific databases, the transaction will not protect interaction among them. One workaround is to begin a transaction on each class whose models you alter:

```
Student.transaction do
  Course.transaction do
    course.enroll(student)
    student.units += course.units
  end
end
```

Not a great solution... But ActiveRecord is not meant for fully distributed transactions!

Transactions with Active Record objects

Both `#save` and `#destroy` come wrapped in a transaction that ensures that whatever you do in validations or callbacks will happen under its protected cover.

So you can use validations to check for values that the transaction depends on or you can raise exceptions in the callbacks (including `after_*` ones) to roll back.

Exceptions thrown within a transaction block will be propagated after rolling back, so you should be ready to catch those in your application code.

You can learn about exceptions in Ruby here:
https://docs.ruby-lang.org/en/2.7.0/syntax/exceptions_rdoc.html

ActiveRecord Query Interface

“How to find records using a variety of methods and conditions”

https://guides.rubyonrails.org/v6.1/active_record_querying.html

Supports DB queries in a way that is agnostic to the underlying DBMS

Basic tasks include retrieving a single object or multiple objects [in batches].

These slides will cover some of the most commonly used methods and features.

ActiveRecord Query Interface

`find`

Using the `find` method, you can retrieve the object corresponding to the specified primary key that matches any supplied options.

```
# Find the customer with primary key (id) 10.
irb> customer = Customer.find(10)
=> #<Customer id: 10, first_name: "Ryan">
```

`ActiveRecord::RecordNotFound` exception if no matching record is found.

You can also query for multiple objects. Call it and pass in an array of primary keys: the method will return an array containing all of the matching records.

ActiveRecord Query Interface

`first`

It finds the first record ordered by primary key:

```
irb> customers = Customer.first(3)
=> [#<Customer id: 1, first_name: "Lifo">, #<Customer id: 2,
first_name: "Fifo">, #<Customer id: 3, first_name: "Filo">]
```

Returns `nil` if no matching record is found (no exception raised)

If invoked on a collection that has been sorted using `order`, it returns the first record according to order implied by the attribute selection.

ActiveRecord Query Interface

`find_by`

It finds the first record matching some condition(s):

```
irb> Customer.find_by first_name: 'Lifo'
=> #<Customer id: 1, first_name: "Lifo">
```

Returns `nil` if no matching record is found. The variant find_by! will instead raise an `ActiveRecord::RecordNotFound` exception.

ActiveRecord Query Interface

`where(*args)`

Returns a new relation, which is the result of filtering the current relation according to the conditions in the arguments. For example:

```
Customer.where(first_name: 'Lifo').take
```

This will behave like the `find_by` example from the previous slide (beware: we need to call `take` here on the output relation).

https://apidock.com/rails/v6.1.3.1/ActiveRecord/QueryMethods/where

ActiveRecord Query Interface

`find_each`

The method retrieves records in batches and then yields each one to the block. It takes in options for `start`, `finish`, `batch_size`, and errors. For example:

```ruby
Customer.find_each(start: 2000, finish: 10000) do |customer|
  NewsMailer.weekly(customer).deliver_now
end
```

https://apidock.com/rails/v6.1.3.1/ActiveRecord/Batches/find_each

`find_in_batches` instead yields each batch of records that was found by the find options as an array.